

LECTURE 1

Introduction to Operating Systems

CSC323 · Operating Systems

COMSATS University Islamabad, Lahore Campus

In This Lecture

- What an operating system is — from the user's seat, and from the system's
- How a computer system is organized around interrupts, replacing constant polling
- The building blocks of I/O: device controllers and direct memory access (DMA)
- Computer system architectures — single-processor, multiprocessor, and clustered
- Multiprogramming, multitasking, and the dual-mode (and multimode) execution model

What Is an Operating System?

The one program that runs at all times, sitting between every user program and the hardware, controlling and coordinating its use. Two very different, equally valid answers exist for “what does it do?”

User view

Make the computer convenient and pleasant to use — maximize ease of use, not necessarily efficiency

System view

Allocate scarce hardware resources fairly and safely among every competing program

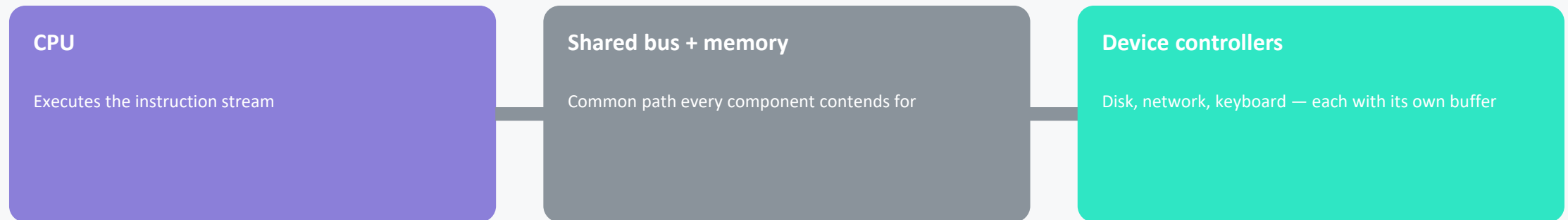
An embedded OS inside a microwave, engine controller, or ATM has effectively no human “user” — it just runs, with no interface to judge convenience by.

The System View: Resource Allocator & Control Program

- **Resource allocator** — a computer has many resources (CPU time, memory, storage, I/O devices) multiple programs may need at once; the OS decides who gets what, when, and for how long
- **Control program** — manages execution of every user program specifically to prevent errors and improper use of the computer
- **Protects the system** — one misbehaving program must never corrupt another's memory, monopolize the CPU, or crash the machine for everyone else
- **No single definition fits every OS** — “general-purpose OS” isn't even precisely defined; what matters is the services it provides and the protection it enforces

Computer System Organization

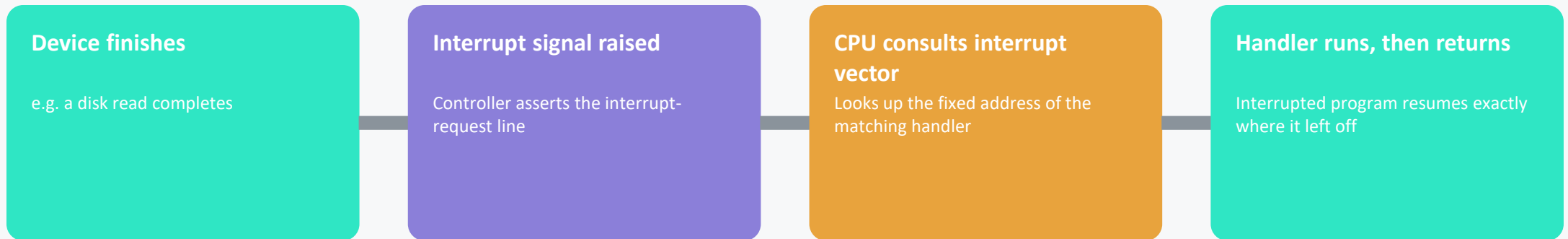
Strip away the OS and applications, and a modern computer is one or more CPUs plus device controllers, connected through a shared bus to memory — all competing for memory cycles. Something has to keep the CPU and devices synchronized without wasting CPU time watching them.



That synchronizing mechanism is the interrupt.

Interrupt-Driven I/O

The alternative is polling: a loop that repeatedly checks a device's status register until it's ready, burning CPU cycles doing nothing. Interrupt-driven I/O flips this — a device tells the CPU “I'm done” instead of being asked.



I/O Structure: Device Controllers & DMA

- **Device controller** — hardware in charge of one device class, with its own local buffer and registers; it moves data between the device and that buffer, not the CPU directly
- **Fine for slow devices** — having the CPU handle every byte from a keyboard is no real burden
- **Ruinous for fast devices** — a disk or NIC would tie the CPU up babysitting the transfer instead of computing
- **Direct memory access (DMA)** — a dedicated controller moves an entire block directly to/from memory; the CPU is interrupted once per block, not once per byte

Computer System Architecture: Single-Processor

- The simplest, historically most common case: one general-purpose CPU running the system's main instruction stream
- Many single-processor systems also contain special-purpose processors — a disk controller's own chip, a graphics processor
- Special-purpose processors run a very limited instruction set, never run user programs, and don't count as “the” processor for scheduling

Multiprocessor Systems

Two or more CPUs in close communication, typically sharing one bus, clock, memory, and peripherals (“tightly coupled”). Three reasons organizations invest in them:

- **Increased throughput** — more processors complete more work in the same time, though speedup is never perfectly linear
- **Economy of scale** — one multiprocessor machine sharing peripherals and power costs less than N separate machines
- **Increased reliability** — failure of one processor doesn't halt the system — graceful degradation / fault tolerance
- **Asymmetric vs. symmetric (SMP)** — fixed master/slave roles, or every processor a peer; SMP dominates today, including ordinary laptop and phone multicore chips

Clustered Systems

- **Loose coupling** — two or more complete computer systems, each with its own CPU(s), memory, and OS, coupled over a LAN or fast interconnect
- **Shared storage via a SAN** — a storage-area network lets any node reach any disk, enabling high availability if one node fails
- **Asymmetric or symmetric** — one node in hot standby, or several nodes all running applications while monitoring each other
- **Load balancing** — distributing work across nodes also provides a performance benefit

Single-Processor vs. Multiprocessor vs. Clustered

Single-processor

One general-purpose CPU, plus special-purpose controllers that never run user programs

Multiprocessor

Tightly coupled — several CPUs share one bus, clock, memory, and peripherals

Clustered

Loosely coupled — independent computers, each with its own OS, sharing storage over a SAN

Not the same thing

A multiprocessor system's CPUs share one memory and one OS instance over a shared bus; a cluster's nodes each run their own OS and communicate over a network, sharing only storage.

Multiprogramming and Multitasking

The earliest systems ran one job at a time; the CPU sat idle whenever that job waited for I/O.

Multiprogramming

Keeps several jobs in memory as a job pool; the instant one waits on I/O, the OS switches to another — goal: CPU utilization

Multitasking / timesharing

Switches between jobs so fast that users can interact with each one — goal: responsiveness, typically well under a second

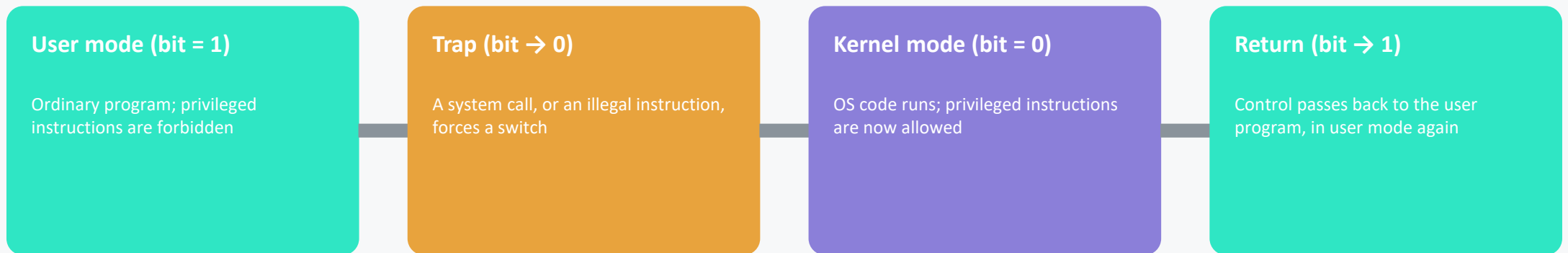
A program loaded into memory and executing is called a process. Multitasking is built on top of multiprogramming's job-switching idea, but the two solve different problems.

Dual-Mode Execution

Sharing one CPU among independent programs is dangerous unless hardware enforces limits — a software policy alone cannot guarantee protection.

- **Two hardware modes** — user mode and kernel mode (also: supervisor / system / privileged), tracked by a single mode bit — 0 for kernel, 1 for user
- **At boot** — hardware starts in kernel mode; the OS loads while still in kernel mode; the bit flips to user mode once user processes begin
- **Privileged instructions** — controlling I/O, the timer, interrupts, memory protection — only executable when the mode bit says kernel mode
- **The only sanctioned doorway** — a user program asks the OS to act on its behalf only through a system call (Lecture 3)

The Dual-Mode Execution Cycle



If a user-mode program attempts a privileged instruction anyway, the hardware itself refuses and traps into the OS — this is the entire basis of OS protection.

Multimode Execution

Dual mode was enough when “trusted OS” and “untrusted program” were the only two categories. Modern hardware, driven by virtualization, adds a third privilege level.

Think of it as more rings, not a different idea

Picture privilege as concentric rings: user programs outermost, the OS kernel one ring in, and — on virtualized hardware — a hypervisor one ring further in. Hardware extensions like Intel VT-x or AMD-V let the hypervisor trap privileged instructions from a guest kernel without that kernel ever knowing it isn't running on bare metal.

The hypervisor manages one or more guest operating systems running as virtual machines — the mechanism is identical to dual mode's bit; multimode just adds more levels.

Key Takeaways

- An OS has two faces: the user view (convenience) and the system view (resource allocator + control program)
- Interrupts let the CPU react to devices instead of polling; the interrupt vector gives constant-time dispatch to the right handler
- Device controllers move data to/from their own buffers; DMA lets large transfers happen without the CPU babysitting every byte
- Single-processor, multiprocessor (tightly coupled), and clustered (loosely coupled, shared storage via SAN) systems trade off cost, throughput, and reliability differently
- Multiprogramming keeps the CPU busy across a job pool; multitasking/timesharing switches fast enough to feel interactive
- Dual-mode execution — a hardware mode bit — is what actually enforces OS protection; multimode extends the idea to support hypervisors